Simplifying the K215 Correctness Proofs

Semantic coverage, a shared implication core, and measured priorities

Prepared for Vladimir Reshetnikov's ProveIt project

21 October 2026

Abstract

The end-to-end correctness statements for K215 are already short; the substantial simplification opportunities are in the interfaces and repeated certificates underneath them. This report reviews a pinned repository snapshot and supplies two concrete developments. First, a Lean refactor separates semantic edge correctness from the Boolean symbolic-execution check that currently supplies it, allowing shared primitive proofs and unchanged checked leaves to coexist. Second, a Rocq backend reuses the repository's existing 58-step representation-independent dominated-implication derivation through a small numerical adapter. The latter changes proof organization and removes an unnecessary truth premise from the generic argument; it does not discover a new 58-step derivation. A finite abstract-reachability experiment shows limited payoff for the tested nonrelational value domains: they remove only five of 6,176 current site obligations. The report states the mathematical contracts, explains why positive physical progress is essential, distinguishes focused compilation from endpoint verification, and gives an incremental integration plan. The literal 215-state machine is unchanged.

Contents

1 Recommendation and scope **2**

2 Pinned baseline and what has already been done **2**

3 The main barrier: Boolean coverage is too specific **4**

4 Why the semantic interface is sufficient **5**

5 A common implication core for Lean and Rocq **7**

6 Reachability pruning: a useful negative result **9**

7 Verification boundary and integration plan **10**

8 Further research questions **11**

1 Recommendation and scope

The next substantial simplification should make the runtime simulation depend on *what each source instruction does*, with the existing Boolean checker retained as one producer of that evidence. Once this boundary exists, a decoder theorem, a primitive theorem and a return theorem can replace repeated whole-edge symbolic executions. This is a change to the proof interface, not to the machine, enumerator or consistency calibration.

In parallel, the two assistants should consume one representation-independent implication derivation. Lean already has the appropriate core. The submitted Rocq backend and adapter provide a concrete way to bring the maintained Rocq proof to the same abstraction boundary. Keeping numerical coding outside the generic derivation makes its logical assumptions visible and reduces the amount of code that must be reviewed twice.

This report addresses the request to shorten and simplify the correctness proofs. It does not conduct a search for a smaller machine. In particular, none of its counts or certificate reductions changes the 215 working states, 430 transition actions, startup convention, or input tape.

There are three different outcomes in the packet:

1. A concrete Lean interface refactor, with backwards-compatible theorem wrappers and mixed semantic/checker coverage.
2. A concrete Rocq rendering of an existing abstract derivation, plus a numerical compatibility adapter.
3. An executable, explicitly non-formal experiment that helps prioritize the next stage.

The verification ledger accompanying the source files is authoritative for exactly what was compiled in this delivery. A focused module check is not described as a fresh verification of the entire K215 endpoint.

2 Pinned baseline and what has already been done

The review is based on repository commit

`2ca3a5c16f0876d2cab842056353c03a1ed2f5f0`.

All comparisons refer to that snapshot, not to an unspecified moving `main`. The repo was obtained through the selected GitHub connection and inspected in a detached local checkout. No remote branch or pull request was changed.

The exact machine identities agree with the repository's verification record [1]:

Input	SHA-256
Dense table `k215.tm`	`fdbcef0825886035c3091c8537a69535bc96f9e265f0ae45863543d40ae70090`
Native table	`9683d6ed9628e6ce407c315b08f5da3d063da94e92706f275ca0f33e8272e4b2`
Source `sites215.json`	`17af207195179b2508bf91154cd00739374f8862207ca21b9a3cb9098aa6f5ba`

2.1 Existing proof architecture

Both lanes compose the following mathematical layers: the dense table is related to the sparse table; initialization reaches a dispatch configuration; the binary machine simulates a register program; the literal source implements the intended enumerator; the enumerator finds exactly the relevant finite refutations; and the compact calculus has the required consistency calibration with ZFC. The Lean theorem `KEndpoint215.blank_halts_iff_zfc_refutation` instantiates a generic endpoint theorem. The Rocq dense endpoint applies dense/sparse halting equivalence and the sparse endpoint [4, 5].

There is consequently little to gain by shortening these final compositions themselves. The pinned Lean endpoint is only 54 lines including documentation and its assumption report; its generic composition module is 80 lines. The interesting work lies below these wrappers.

The repository already contains extensive simplification research. In particular:

- The dominated-implication argument has already been separated from numerical coding in Lean. Its current core has 58 condensed-detachment steps, 295 lines and 13,984 bytes, with a 73-line, 2,680-byte numerical adapter [6, 7, 8].
- Shared seed semantics already replace duplicated semantic arguments. They should not be presented again as a new result.
- Phase nine already shares the positive Home-return theorem and its finite table gate. It records all nine site batches checked, with the same 4,751 leaves and 6,176 branch outcomes [2].
- The exact source-to-decoder-entry bridge is already checked. Its endpoint retains the whole source prefix and reaches a precise primitive-entry zipper, but stops before the primitive action [10].
- Normalized source correspondence, balanced data access, framed decoder proposals, positive-stuttering arguments and several earlier transport proposals are already present. Their existence is a reason to finish the missing interfaces, not to restart the same investigation.

2.2 Merged source is stronger evidence than a commit summary

The current Lean core and the maintained Rocq file are not at the same stage. At the pinned commit, Rocq's `coq/215/DomRho8.v` still contains the reflective certificate: 1,169 lines, 22,673 bytes, 735 instructions and 272 symbolic DAG entries. The 52-step numerical Rocq rendering is present as upstream research and in historical commits. It has not replaced the maintained production file in this snapshot [9].

Similarly, historical phase-nine documents describe their own checkpoint, which predates the later generic-core merge. The new 58-step Lean core has focused build evidence in the later history; that fact is not a claim that this delivery rebuilt the 192-module endpoint closure. The report therefore keeps source inspection, inherited receipts, new focused compilation and proposed integration separate.

Phase nine records 984.608 seconds for its nine site batches, compared with 1,649.340 seconds in a historical phase-six run. The repository explicitly says this is not a fresh controlled paired benchmark. It also says the site objects were byte-identical: the improvement reduced conversion work, not object size [2]. Those distinctions should be retained when evaluating the next refactor.

3 The main barrier: Boolean coverage is too specific

In the pinned `KSim.lean`, the global coverage hypothesis is essentially

$$\forall P\ \exists e, d, \quad \mathrm{walk}(p, P) = \mathrm{some}(e, d) \quad \wedge \quad \mathrm{checkEdge}(M, \mathrm{consume}(P, d), e) = \mathrm{true}\,.$$

The `checkEdge` function checks path length, register bounds and a symbolic execution with fixed fuel. Its correctness theorem turns acceptance into the appropriate concrete run [3]. The checker is sound and already useful. The problem is that the *simulation interface* requires that particular acceptance equation.

Suppose a new shared theorem proves exactly the concrete behavior of a whole family of primitive operations. Such a theorem cannot directly supply an equality saying the old executable checker returns `true`. Re-executing the checker to recover that equality loses the intended saving. A separate semantic interface is therefore the prerequisite for using shared proofs efficiently.

3.1 The semantic contract

For a source word Q and edge e, let $\mathsf{EdgeContract}(M, Q, e)$ record the information actually used by the simulation proof. The contract must retain all relevant conditions: bounds needed to represent the program counter; register-index validity; the two distinct decrement outcomes with their zero/nonzero hypotheses; the precise target tape; and genuine target halting for a halt leaf. A nonterminal contract gives a *positive* concrete run.

The target of an increment is not merely "the register increased." It includes the correct program counter, overflow marker behavior, gap, physical register ordering and residual tape. The source uses nine logical registers; the physical order is

$$[2, 8, 1, 4, 3, 7, 0, 5, 6],$$

with a nonempty extension of spare registers. An increment appends the extra spare block exactly as the existing target representation requires. The semantic refactor preserves these details rather than replacing the concrete contract by an informal high-level description.

Define semantic coverage by replacing the Boolean acceptance clause above with $\mathsf{EdgeContract}$. Then prove the finite-checker adapter once:

$$\mathrm{checkEdge}(M, Q, e) = \mathrm{true} \quad \Longrightarrow \quad \mathsf{EdgeContract}(M, Q, e),$$

under the same register-routine and terminal-state assumptions used by the old soundness proof. The semantic simulation is independent of which certificate producer supplied this contract.

The old `sim_step`, `sim_halt` and `halts_iff` interfaces remain wrappers. The old `sim_stop` interface retains its short structural case analysis because it has never required the register-routine lemmas. This preserves existing endpoint callers while exposing a more general core to future proofs. It also prevents a migration from silently weakening the old all-register theorem.

3.2 Mixed coverage makes adoption incremental

Let $S(Q, e)$ be a decidable selector for a family whose semantic contract has been proved directly. For every leaf of the source tree, check the disjunction

$$S(Q, e) \ \vee \ \mathrm{checkEdge}(M, Q, e).$$

The proof obligations are:

$$S(Q, e) = \text{true} \Longrightarrow \mathsf{EdgeContract}(M, Q, e),$$
$$\text{checkEdge}(M, Q, e) = \text{true} \Longrightarrow \mathsf{EdgeContract}(M, Q, e).$$

The ordinary exhaustive source-tree traversal can then provide semantic coverage. An empty selector recovers the existing route. A selector for one completed primitive family changes only that family. No unproved site is accepted merely because its selector is true: the first implication is a formal proof premise.

This migration design is useful even if the first shared primitive theorem saves little. It avoids a large all-at-once replacement and permits old and new evidence to be compared on the same literal source. It is also portable to Rocq: the theorem interface is a semantic proposition, while each assistant may use a different certificate representation.

3.3 Implemented and checked Lean result

The supplied `lean/KSim-semantic.patch` implements this interface in the actual module. Its new public objects are `RunContract`, `HaltContract`, `EdgeContract`, `SemanticCovered`, `checkEdge_contract`, the generic and mixed finite-coverage lemmas, and the four theorems in `KSim.Semantic`. The original simulation inductions occur once, in that semantic namespace.

The modified module and its twelve project dependencies were freshly compiled with Lean 4.32.0. A separate typed client checks all four old theorem signatures, the new semantic halting theorem and the mixed-coverage interface. A second standalone replay from the bundled source snapshots also passed. Twelve named axiom reports use only the standard `propext`, `Classical.choice` and `Quot.sound`; the semantic step, halt and halting-equivalence theorems use only `propext` and `Quot.sound`. This local separation of dependencies does not remove choice from the complete K215 endpoint.

All original executable definitions are preserved, as are the first 9,648 bytes of the module through `rel_view`. The source grows from 356 lines / 16,146 bytes to 485 lines / 21,988 bytes: an explicit overhead of 129 lines / 5,842 bytes. No concrete K215 leaf check has yet been replaced. This is checked enabling infrastructure, not an immediate global source reduction or a measured runtime improvement. The source and typed audit have no warnings; inherited dependency linters are retained in their raw logs.

4 Why the semantic interface is sufficient

This section gives the mathematical argument independently of implementation notation. It explains the exact obligations that a shorter proof must preserve.

Let S and T be deterministic partial transition systems with step functions s and t. Let H_S designate the intended source halt configurations; an undefined source step may also represent a malformed program, so it must not automatically count as an intended halt. Let $R(c, C)$ relate a source configuration and a concrete target configuration. Write $C \to_T^k C'$ for a successful k-step target execution and $\text{Halts}_T(C)$ for a finite execution reaching a configuration with undefined target step.

Theorem 4.1 (Positive simulation with explicit halt classification)**.** *Suppose the following hold whenever* $R(c, C)$*.*

1. *If* $s(c) = \text{some}(c')$*, then for some* C' *and* $k \geq 1$*,* $R(c', C')$ *and* $C \to_T^k C'$*.*
2. *If* $H_S(c)$*, then* $\text{Halts}_T(C)$*.*
3. *If* $s(c) = \text{none}$*, then* $H_S(c)$*.*

Then the source has a finite execution from c *ending in* H_S *if and only if* $\text{Halts}_T(C)$*.*

Proof. For the forward implication, induct on the length of the successful source execution. At length zero, use condition 2. At a successor, condition 1 gives a target run to a related configuration, apply the induction hypothesis there, and concatenate runs.

For the reverse implication, use strong induction on the length n of a target execution to its first terminal configuration. If $s(c)$ is undefined, condition 3 supplies the intended source halt immediately. Otherwise condition 1 supplies a source successor and a successful target run of length $k \geq 1$. Necessarily $k \leq n$: determinism makes the two target executions coincide until one ends, and a successful execution cannot continue past a configuration with undefined step. Removing the common k-step prefix leaves a target halting execution of length $n - k < n$. The induction hypothesis yields a source halting execution from the successor; prepend the source step. □

The current `KSim.halts_iff` already uses this argument. The refactor preserves the induction and changes the local supplier of its premises. The positive run is not a technical preference. If zero-step matching were allowed without an additional well-founded rank, a source self-loop could be related forever to one already halted target state. Step preservation would hold by reflexivity while halting equivalence failed.

The explicit stop-classification premise matters too. An invalid decoder result or a node edge is not a refutation found by the enumerator. Coverage must exclude such stuck cases, or classify them separately; otherwise a proof about "undefined source step" can accidentally prove the wrong halting statement.

4.1 Composing the decoder, primitive and return

The checked decoder-prefix theorem already gives a source-determined entry port with a successful run of length $d \geq 0$. A shared primitive theorem must start at that exact port and reach a continuation configuration; a return theorem completes the desired target representation. If their respective durations are d, p, r with $p \geq 1$ and $r \geq 0$, composition has length

$$d + p + r \geq 1.$$

Thus the decoder prefix itself need not be positive. This matches the repository's explicit status: the prefix may have a zero clock because it stops before the primitive action.

The contract must preserve the entire tape, not merely the active register block. In particular, the consumed prefix remains on the left of the zipper, the marker and gap remain in the specified positions, and any untouched frame must remain unchanged. This is exactly why sharing solely by the current machine state and scanned bit is insufficient. The repository's context-aware decoder experiment records a counterexample to that shortcut [13].

Quantifier order is another useful review test. The exact decoder port should have the form

$$\exists\, port\ \forall n, registers\ \exists d, \quad \mathsf{run}(input(n, registers), d) = port(n, registers),$$

with the control state and full source prefix fixed before the arbitrary data. Choosing a different control port after seeing the register values would make a shared residual theorem much harder to apply and could conceal an incorrect context abstraction. The existing `entry_uniform` theorem supplies the stronger order [10].

5 A common implication core for Lean and Rocq

The numerical calculus grants the schema

$$\rho(a,b,c,u) = ((c \Rightarrow u) \Rightarrow a) \Rightarrow ((a \Rightarrow c) \Rightarrow (b \Rightarrow c))$$

only when $u \preceq a$. Its calibration needs every unrestricted instance. The maintained Lean core already proves this using an abstract carrier A, a binary operation $\Rightarrow$, and a relation $\preceq$ satisfying just

$$\begin{aligned} x &\preceq x, \\ u \preceq x &\Longrightarrow u \preceq (x \Rightarrow y), \\ u \preceq y &\Longrightarrow u \preceq (x \Rightarrow y). \end{aligned}$$

For a predicate $D : A \to \mathsf{Prop}$, the derivability hypotheses are modus ponens and the restricted row:

$$\begin{aligned} D(a \Rightarrow b) \wedge D(a) &\Longrightarrow D(b), \\ u \preceq a &\Longrightarrow D(\rho(a,b,c,u)). \end{aligned}$$

No truth constant, numerical zero, injectivity of coding, antisymmetry or transitivity of $\preceq$ is required by this generic derivation. The current 58-step data also supplies composition and prefixing:

$$\begin{aligned} &D((a \Rightarrow b) \Rightarrow ((b \Rightarrow c) \Rightarrow (a \Rightarrow c))), \\ &D((a \Rightarrow b) \Rightarrow ((c \Rightarrow a) \Rightarrow (c \Rightarrow b))). \end{aligned}$$

Modus ponens turns the latter into the familiar right-lifting inference.

5.1 What the submitted Rocq backend changes

The submitted renderer consumes the *unchanged* repository file `derivation_core.txt` and produces direct Rocq lemmas over the abstract interface. The proof search and its derivation are inherited work. The contribution here is the checked generic Rocq backend, its API-compatible numerical adapter, and the verification evidence for this focused replacement.

The adapter instantiates $A = \mathbb{N}$, $\preceq = \leq$ and $\Rightarrow = \texttt{wim}$. The numerical coding lemmas showing that each implication operand is bounded by its code discharge the two insertion conditions. The production `DomImp` record can retain its truth field for compatibility with current callers, while the adapter deliberately does not use that field to obtain unrestricted ρ. This is a small but meaningful reduction in the hypotheses of the mathematical core.

The final adapter also avoids arithmetic automation: it composes the existing pairing bounds with elementary order lemmas. The measured source comparison is:

Rocq implication component	Lines	Bytes
Maintained reflective replay	1,169	22,673
Submitted abstract core	281	14,252
Submitted numerical adapter	36	1,374
Submitted combined total	317	15,626

The reduction is 852 lines (72.9%) and 7,047 bytes (31.1%), measured over the proof sources only. The generator, input derivation, audit and evidence packet are additional engineering artifacts and are not included in that source comparison.

The earlier 52-step numerical Rocq rendering is smaller still: its documented size is 11,190 bytes [8]. It is an appropriate alternative when minimizing that one file is the main objective. The present 58-step backend emphasizes a common abstract specification, elimination of the truth premise, and the additional composition/prefixing interfaces. It is shorter than the maintained reflective replay, not the smallest known rendering of the row alone. This report claims no minimality for either proof.

5.2 Focused Rocq verification

The seven-module calculus slice through the unchanged production `K215F.ThmDom` client compiles, as does an explicit public-type probe. Recursive `coqchk` passes for that slice; the generic core's four reports, the adapter and the calculus client are closed under the global context, and the recursive report lists no axioms or unsafe fixpoints. The existing `rho8_all` theorem type and `DomImp` record interface are preserved.

These checks used the installed Coq 8.18.0 with OCaml 4.14.1. In private build copies only, imports beginning `From Stdlib Require` are changed to `From Coq Require`. The supplied production-format sources retain `Stdlib`; both original and compiled-source hashes are recorded. A fresh build on the repository's production Rocq 9.1.0 toolchain and the complete binary-machine/ZFC endpoint remains an integration gate. This delivery does not describe the compatibility run as that production build.

The previous reflective proof has a soundness development for expressions, substitution, term equality, a symbolic DAG, instruction evaluation and a final Boolean certificate check. Such reflection is useful for large certificates. For this already short derivation, direct abstract lemmas make the argument easier to inspect. Both methods remain kernel checked; replacing reflection does not remove either assistant's trusted kernel or turn an untrusted generator into an axiom.

5.3 What must remain shared, and what may differ

The shared source should be the derivation data and its documented public goals, with separate renderers for the assistants. The record layouts, implicit-argument conventions and proof terms need not be textually identical. The important requirements are reproducibility, unchanged theorem types and checked instantiations.

Blindly sharing generated files across toolchains is less useful than sharing the mathematical specification. Numerical adapters should stay small. The old record and exported theorem names should remain until their clients are deliberately migrated. A generator change must be included with a generated-source change; otherwise a future regeneration can silently restore the superseded proof.

6 Reachability pruning: a useful negative result

One possible shortcut is to prove only the register branches reachable from the actual boot state. The existing site contracts quantify arbitrary admissible register contents independently at each site. That universality is stronger than the blank-tape endpoint strictly needs, so a source invariant could, in principle, eliminate impossible decrement outcomes.

The packet includes an executable finite experiment on the exact source. It reconstructs the 3,326-primitive control-flow graph and uses a bounded-value abstraction: values below a chosen cap are distinguished, with a final interval for all larger values. It compares caps $1, 2, 4, 8, 16$ and three assumptions about the enumeration counter: arbitrary, positive, and the abstraction of its concrete initial value 4. Other registers start at zero. Every resulting invariant is checked again for the post-fixpoint property. The source hash and the comparison with the archived primitive CFG are recorded in its output.

The actual logical boot register file is $(4, 0, \ldots, 0)$, as defined by `L2Perm.bootRegs` in `SeedPerm.lean`. Under the physical order above it becomes $(0, 0, 0, 0, 0, 0, 4, 0, 0)$ followed by 40 spare one-blocks in `KBoot215.bootVs`; `KEndpoint215.boot_rel` binds these descriptions [11, 12, 4]. Older examples with another parameter initially equal to 3 belong to other machines. For caps at most 4, the class containing 4 overapproximates that singleton; for caps 8 and 16 it is exact.

The observed result is small:

Abstraction	Remaining obligations	Removed
Zero/nonzero	6,173	3
Caps $2, 4, 8, 16$	6,171	5
Current all-register baseline	6,176	0

The stronger tested abstractions remove three dead sites and two decrement outcomes, about 0.081% of the current obligations. These are experiment results, not a kernel-certified elimination of those branches and not a lower bound on what a relational invariant could achieve.

For reproducibility, the excluded positive decrement outcomes are zero-based leaf 4671 (register 1, path `11010010111000`) and leaf 4733 (register 6, path `1111001100`). The dead leaves are 4672 (increment register 6), 4673 (continue 2) and 4734 (continue 2). The first of those branches and its descendants were already observed in prior search discussion. This delivery's contribution is the measured comparison across fifteen runs of the simple abstraction family, not the discovery of every individual dead branch. An independent rerun during preparation reproduced the full JSON result exactly.

The logical obligation for a future formal version is straightforward. If $I(c_0)$ holds at the exact source boot configuration, and $I(c) \wedge c \to c'$ implies $I(c')$, induction proves I on every finite run. Contracts restricted to I then suffice for the designated boot endpoint. They do not justify replacing the original all-configuration theorem by a weaker statement under the same name.

Given the measured yield, simple value-domain pruning should not be the next engineering priority. Sharing a primitive proof can remove repeated reasoning even when every branch remains feasible. More sophisticated relational invariants remain possible, but their extra formal development should be justified by a concrete census before adoption.

7 Verification boundary and integration plan

The packet contains source changes and focused evidence; the final build ledger gives the exact compiler versions, commands, source digests, outputs and assumption reports. The new results should be integrated in the following order.

1. Apply the Lean interface patch to the pinned baseline, compile its fresh dependency closure, and run typed clients for both the retained public API and the semantic API. Verify that executable definitions and theorem types promised to remain unchanged actually do so.
2. Generate the abstract Rocq file from the unchanged shared derivation, compile the core and numerical adapter, and check the current calculus client. Use the production Rocq toolchain for the eventual maintained-tree gate; a compatibility check on another installed version is recorded separately.
3. Update the maintained generators and manifests in the same integration change. Preserve the literal table/source hashes above.
4. Prove one primitive family from the exact decoder port through its continuation and return, and use a nontrivial mixed-coverage selector for just that family. Keep the old checker for all remaining leaves.
5. Rebuild every affected source descendant, then both endpoint clients. Reuse an old object only with exact transitive source and dependency bindings. An unchanged object hash alone is not sufficient evidence after executable dependencies change.
6. Compare the old and new primitive-family proof on the same machine, toolchain and source corpus. Record wall time, peak memory, source size and compiled size separately. Only then state an endpoint-wide speed or size improvement.

The recent repository history already contains a rejected combined run whose byte-identical intermediary objects masked a transitive executable-definition change. Its corrected policy is relevant here [2]. The purpose of exact bindings is to ensure that the proof being reported is the one whose current definitions were actually compiled.

7.1 Files supplied for review and replay

The archive is self-contained for the focused checks once the indicated compilers are installed. It contains this LaTeX source and its compiled PDF; the Lean patch, candidate, baseline, thirteen-module source snapshot and typed audit; the Rocq backend, unchanged derivation data, adapter, seven-module source slice and public-type probe; source/table inputs; the fifteen-run reachability experiment; raw logs, manifests, source pins and attribution notices. Compiled proof objects are represented by their recorded hashes rather than included as a substitute for rebuilding.

`README.md` gives exact replay commands. `verify_bundle.py` checks the delivered byte identities; it is an integrity check, not a proof checker. The `lean/` and `rocq/` subdirectories contain their own reproducibility drivers. Re-running the finite Python experiment does not refresh either assistant's proof receipts.

7.2 External correctness versus an internal ZFC theorem

The checked endpoint statements are mathematical theorems in Lean and Rocq:

$$\text{Halts}(K215, \textit{blank}) \quad \longleftrightarrow \quad \text{ZFC has a finite derivation of falsity.}$$

Negating the equivalence gives nonhalting iff syntactic consistency. Neither direction assumes that ZFC is consistent. The user-facing goal of an equivalence provable *in ZFC* is compatible with this program, but the repository explicitly distinguishes the external assistant theorems from an internal PA proof or a derivation encoded in ZFC's own syntax [1]. This delivery does not close that separate formalization boundary.

Likewise, it does not prove unconditional nonhalting. An equivalence between nonhalting and Con(ZFC) supplies no proof of either side without further assumptions. There is no need to weaken the consistency target or introduce arithmetic soundness merely to simplify these proof modules.

8 Further research questions

1. Can one complete a primitive/continuation family with sufficiently few parameters that the semantic contract replaces a substantial block of the current 6,176 outcomes? The first measurement should be for a real selected family, including its full left prefix and right frame.
2. Can semantic coverage become the single stable runtime API in both assistants while their finite checker implementations evolve independently? A useful criterion is that the same mathematical edge specification admits both a direct proof and a generated certificate.
3. Which context is truly necessary for a reusable primitive theorem? State and scanned bit alone are known to be too weak; the full prefix is safe. A certified quotient of prefix contexts could provide a middle ground, but only after proving its continuation behavior.
4. Can a source-structured recursive proof replace repeated path-indexed obligations once primitive contracts exist? The target would be one theorem per distinct source constructor or verified context class, not an unsupported inference from the number of distinct syntax subtrees.
5. Would a relational register invariant eliminate substantially more than the five outcomes removed by the tested nonrelational domains? A positive finite census should precede an invariant-indexed endpoint development.
6. Can the representation-independent implication core be reused by future numerical encodings without any change to the derivation? The adapter boundary makes that question concrete: only operand domination and the coding of the row should need to be reproved.
7. Can the external consistency equivalence be internalized with a smaller syntactic proof-translation interface, avoiding an unnecessary repeat of the full semantic model development? This is distinct from ordinary module simplification and should have its own explicit target statement.

The most useful next milestone is a complete semantic contract for one real primitive family, installed through mixed coverage and measured against the same old checks. That milestone tests the architecture at the point where a shorter global proof can actually emerge.

References

[1] ProveIt, K215 formal verification record, pinned snapshot. https://github.com/VladimirReshetnikov/ProveIt/blob/2ca3a5c16f0876d2cab842056353c03a1ed2f5f0/Computability/BusyBeaver/Research/ZFC/k225-formal/verification/k215/README.md.

[2] ProveIt, Ninth K215 proof-simplification checkpoint. https://github.com/VladimirReshetnikov/ProveIt/blob/2ca3a5c16f0876d2cab842056353c03a1ed2f5f0/Computability/BusyBeaver/Research/ZFC/k225-formal/verification/k215/simplification/phase9/README.md.

[3] ProveIt, `KSim.lean`. https://github.com/VladimirReshetnikov/ProveIt/blob/2ca3a5c16f0876d2cab842056353c03a1ed2f5f0/Computability/BusyBeaver/Lean/BusyBeaver/ZFC/KSim.lean.

[4] ProveIt, `KEndpoint215.lean`. https://github.com/VladimirReshetnikov/ProveIt/blob/2ca3a5c16f0876d2cab842056353c03a1ed2f5f0/Computability/BusyBeaver/Lean/BusyBeaver/ZFC/KEndpoint215.lean.

[5] ProveIt, `CorrectDense215.v`. https://github.com/VladimirReshetnikov/ProveIt/blob/2ca3a5c16f0876d2cab842056353c03a1ed2f5f0/Computability/BusyBeaver/Research/ZFC/k225-formal/rocq/rocq-extension/coq/215/CorrectDense215.v.

[6] ProveIt, `DominatedImplication.lean`, existing abstract 58-step core. https://github.com/VladimirReshetnikov/ProveIt/blob/2ca3a5c16f0876d2cab842056353c03a1ed2f5f0/Computability/BusyBeaver/Lean/BusyBeaver/ZFC/DominatedImplication.lean.

[7] ProveIt, `DomRho8.lean`, numerical adapter. https://github.com/VladimirReshetnikov/ProveIt/blob/2ca3a5c16f0876d2cab842056353c03a1ed2f5f0/Computability/BusyBeaver/Lean/BusyBeaver/ZFC/DomRho8.lean.

[8] ProveIt, rho8 by condensed detachment: derivations, renderer and search history. https://github.com/VladimirReshetnikov/ProveIt/blob/2ca3a5c16f0876d2cab842056353c03a1ed2f5f0/Computability/BusyBeaver/Research/ZFC/k225-formal/rho8cd/README.md.

[9] ProveIt, maintained status of the condensed-detachment proposal. https://github.com/VladimirReshetnikov/ProveIt/blob/2ca3a5c16f0876d2cab842056353c03a1ed2f5f0/Computability/BusyBeaver/Research/ZFC/k225-formal/rho8cd/PROVEIT_STATUS.md.

[10] ProveIt, `KDecoderPrefix215.lean`. https://github.com/VladimirReshetnikov/ProveIt/blob/2ca3a5c16f0876d2cab842056353c03a1ed2f5f0/Computability/BusyBeaver/Lean/BusyBeaver/ZFC/KDecoderPrefix215.lean.

[11] ProveIt, `SeedPerm.lean`, exact logical boot registers. https://github.com/VladimirReshetnikov/ProveIt/blob/2ca3a5c16f0876d2cab842056353c03a1ed2f5f0/Computability/BusyBeaver/Lean/BusyBeaver/ZFC/SeedPerm.lean.

[12] ProveIt, `KBoot215.lean`, physical bootstrap target. https://github.com/VladimirReshetnikov/ProveIt/blob/2ca3a5c16f0876d2cab842056353c03a1ed2f5f0/Computability/BusyBeaver/Lean/BusyBeaver/ZFC/KBoot215.lean.

[13] ProveIt, context-aware decoder-sharing research. https://github.com/VladimirReshetnikov/ProveIt/blob/2ca3a5c16f0876d2cab842056353c03a1ed2f5f0/Computability/BusyBeaver/Research/ZFC/k225-formal/decoder-sharing-research/README.md.